UNIFAAT 2026.2

08 - React: Gerenciamento de Estados

Prof. Luan Lourenço

Frontend é mais difícil de organizar

```
# backend · ciclo curto
requisição chega
  ↓ lê dados, monta a resposta
  ↓ termina
cada requisição começa do zero;
o estado longo mora no banco
```

```
# frontend · processo vivo
a página fica aberta por horas
  ↓ eventos em qualquer ordem
  ↓ dados chegando a qualquer hora
  ↓ o mesmo dado em vários lugares
fonte da verdade espalhada:
variável, input, classe CSS, localStorage
```

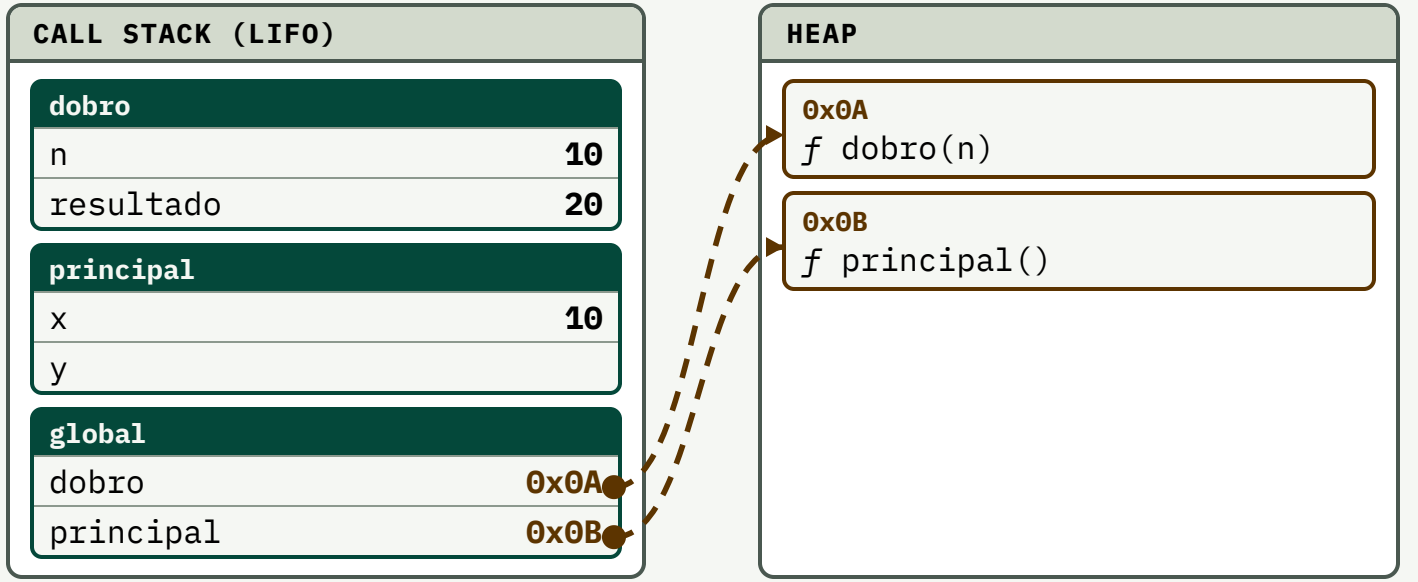
Quanto mais a tela cresce, mais difícil lembrar de "atualizar tudo que depende disso". Esquecer um ponto = tela incoerente.

Call stack: cada chamada de função ganha um frame

CÓDIGO

```
function dobro(n) {
    const resultado = n * 2
    return resultado
}
function principal() {
    const x = 10
    const y = dobro(x)
}
principal()
```

MEMÓRIA DO PROGRAMA



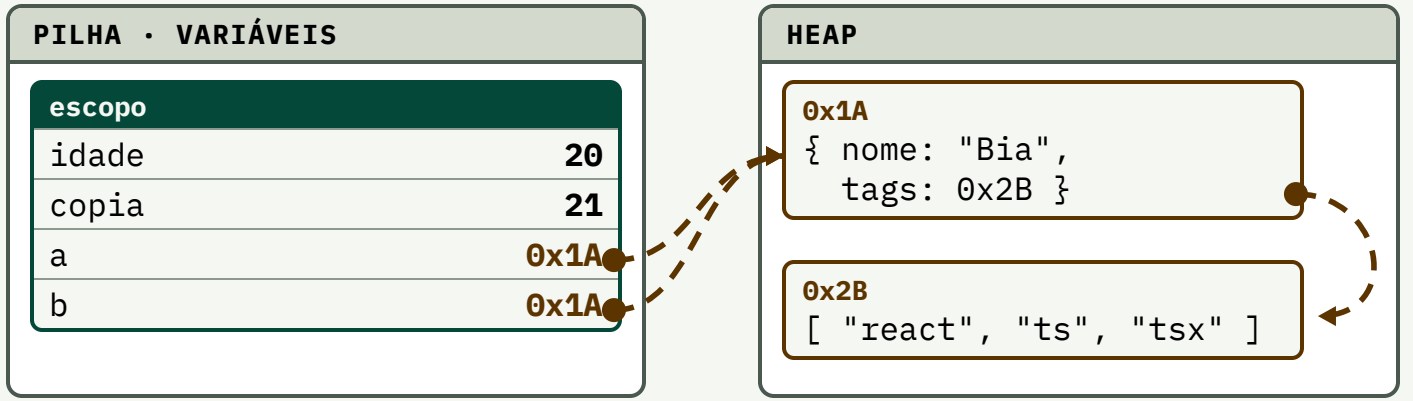
Observação: pilha e heap são regiões da **mesma memória**. A pilha guarda valores e **endereços** (setas); o objeto, inclusive a função, fica no heap.

Heap: copiar valor x copiar endereço

CÓDIGO

```
let idade = 20
let copia = idade
copia = 21
const a = {
    nome: "Ana",
    tags: ["react", "ts"]
}
const b = a
b.nome = "Bia"
b.tags.push("tsx")
```

MEMÓRIA DO PROGRAMA



Observação: primitivo guarda o valor na pilha; objeto guarda só o **endereço** (seta) e vive no heap. Um objeto pode apontar para outro: a propriedade `tags` guarda o endereço do array.

Imperativo x declarativo

```
# imperativo · como mudar a tela
let contador = 0
botao.addEventListener("click", () => {
  contador++
  saida.innerText = String(contador)
})
dois lugares com o número: você sincroniza
```

```
# declarativo · o jeito antigo: o evento na própria tag
<button onclick="somar()">
  Cliques: <span id="saida">0</span>
</button>
o que dispara está declarado no HTML, sem
addEventListener. Era assim antigamente
```

ideia Declarar o evento na tag é **antigo**: o `onclick` do HTML caiu em desuso. O **React voltou com ele**: `onClick={...}`, agora com `UI = f(estado)` e só o que mudou é alterado no DOM.

A proposta: componentes

- **Componente** = uma **função** que recebe parâmetros (`props`) e **retorna** a descrição da tela daquele pedaço
- Pedaços **independentes e reutilizáveis**, encaixados por parâmetros
- O mesmo `TaskItem` serve para qualquer task; só muda o dado que recebe

```
<TaskApp>  ← a tela de tasks
  ├ <TaskForm>
  └ <TaskList>
     ├ <TaskItem>
     ├ <TaskItem>
     └ <TaskItem>
```

Troca de problema: manter dezenas de pontos sincronizados vira descrever um ponto. A sincronização é do React.

JSX e TSX

```
# o return parece HTML; não é
<h1 classNameName="destaque">
  Olá, {task.name}
</h1>
JSX = JavaScript XML: o Vite compila
para chamadas de função. O navegador
nunca vê JSX.
```

Extensão	Linguagem	JSX?
`.js`	JavaScript	não
`.jsx`	JavaScript	**sim**
`.ts`	TypeScript	não
`.tsx`	TypeScript	**sim**

→ `className` (não `class`) · `onClick` em camelCase, recebendo uma **função** · `{ }` abre uma expressão · tags sempre fechadas

---

Temos TypeScript, então usamos **.tsx** nos componentes. Arquivo sem interface continua `.ts`.

Anatomia de um componente

TASKITEM.TSX · CLIQUE NUMA PARTE

```
type TaskItemProps = {                                  ① parâmetros
    task: Task
    onToggle: (task: Task) => void
}
function TaskItem({ task, onToggle }: TaskItemProps) {
    const decoration = task.is_done ? "line-through" : "none"
    return (
        <li onClick={() => onToggle(task)}
            style={{ textDecoration: decoration }}>
            {task.name}
        </li>
    )
}
<TaskItem task={task} onToggle={toggle} />
```

① Parâmetros

Parâmetros. O componente recebe **um único objeto**, o `props`, desestruturado na assinatura e tipado com `TaskItemProps`. É **somente leitura**: o filho lê, o dono é o pai.

O pai injeta a função, a filha executa

PAI · TASKAPP (DONO DO ESTADO)

```
function toggle(task: Task) {
    setTasks(/* marca a task */)
}

return (
  <TaskItem task={task}
    onToggle={toggle} />
)
```

FILHA · TASKITEM (SÓ EXECUTA)

```
function TaskItem({ task, onToggle }) {
  return (
    <li onClick={() =>
      onToggle(task)}>
      {task.name}
    </li>
  )
}
```

IoC **Inversão de controle**: quem decide *o que* acontece é o pai; a filha decide só *quando* chamar. É a mesma ideia do `addEventListener` e dos middlewares do Express.

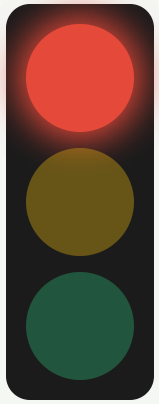
Instalando React + TypeScript

```
# núcleo + ponte com o navegador
npm install react react-dom
# tipos + plugin do Vite
npm install -D @types/react @types/react-dom \
  @vitejs/plugin-react
# projeto novo, já pronto
npm create vite@latest meu-app -- --template react-ts
```

```
# vite.config.js
plugins: [react()]
# tsconfig.json
"jsx": "react-jsx"
"include": ["resources/js/**/*.ts",
  "resources/js/**/*.tsx"]
```

→ `react-jsx` dispensa `import React` em cada arquivo · o HTML precisa de **um** `<div id="root">` onde o React se pendura

Estado: de Alan Turing ao semáforo



estado atual	passa o tempo →
"verde"	"amarelo"
"amarelo"	"vermelho"
"vermelho"	**"verde"**

```
estado = "vermelho"
```

⏱️ passa o tempo

- → **Máquina de Turing (1936)**: fita, cabeçote, tabela de transição e um **estado atual**
- → **Estado** = tudo que o sistema precisa lembrar do passado para decidir o que fazer agora. Mesmo evento, resultado depende do estado
- → **Em orientação a objetos**, o estado são os valores **atuais das propriedades** do objeto instanciado de uma classe: `new Task("Estudar")` começa com `isDone = false`, e `complete()` muda o estado

> **git** Essa foto é o famoso **snapshot**, como um **commit do Git**: guarda o projeto inteiro naquele momento, e o evento leva de um snapshot ao próximo.

Gerenciar estado = responder: quem é o dono, quem lê, quem muda e quem precisa ser avisado.

Estado ou parâmetro mudou: a função roda inteira de novo

PAI · PARÂMETRO

`<Tarefa nome={…} />`

"Estudar React"

"Ler o briefing"

FUNÇÃO TAREFA() · CÓDIGO

```
function Tarefa({ nome }: Props) {
    const [isDone, setIsDone] = useState(false)

    function aoClicar() {
        setIsDone(!isDone)
    }

    return (
        <li onClick={aoClicar}>
            <input type="checkbox"
                checked={isDone} readOnly />
            {nome}
        </li>
    )
}
```

FRAME · TELA

☐ Estudar React

renderizações: **1**

1 · clique no <li>: o navegador chama `onClick`

2 · `aoClicar()` chama `setIsDone`: o React guarda o estado novo

3 · `Tarefa()` roda de novo, inteira

primeira renderização

Clique na tarefa (muda o **estado**) ou troque o texto no pai (muda o **parâmetro**). Nos dois casos o frame volta à pilha e todo o código roda de novo.

Cada renderização vê uma foto do estado

TELA (DOM)

Estudar React — prioridade **0**

renderizações **1**

escolha o jeito de atualizar

`setPriority(priority + 1)`

✓ `setPriority(atual => atual + 1)`

`zerar`

FUNÇÃO TAREFA() · CÓDIGO

```
function Tarefa({ nome }: Props) {
    const [priority, setPriority] = useState(0)

    function aumentarTres() {
        setPriority(atual => atual + 1)
        setPriority(atual => atual + 1)
        setPriority(atual => atual + 1)
    }

    return (
        <li onClick={aumentarTres}>
            {nome} · prioridade {priority}
        </li>
    )
}
```

O QUE O REACT RECEBE

Clique na tarefa. Dentro de uma renderização `priority` é **constante** (a foto): as três linhas veem o mesmo valor. Quando o novo estado depende do antigo, **use sempre o segundo jeito**: a forma de função.

Generics nos componentes

```
# 1 · useState<S>: o tipo do estado (inferido ou explícito)
useState(0)                → number (inferido)
useState<Task[]>([])       → sem o generic, [] vira never[]
useState<Task | null>(null)
# 2 · parâmetros: o type Props
function TaskItem({ task, onToggle }: TaskItemProps)
# 3 · componente genérico
function Lista<T>({ itens, renderItem }: ListaProps<T>)
<Lista itens={tasks} renderItem={t => t.name} /> ← T = Task
```

→ O TypeScript protege o estado: `setCor("azul")` não compila; `selecionada.name` só depois de provar que não é `null`

Parâmetros e estado são as entradas; o código roda a cada renderização; o JSX é a saída.

A View EJS entrega, o React reage

1. Navegador pede a página de tasks
2. Middleware de sessão: sem token, redireciona
3. Controller lê as tasks no banco e chama `res.render("tasks")`
4. EJS devolve o HTML com `#root` e `data-tasks`
5. `createRoot(root).render(<TaskApp initialTasks />)`
6. Clique → API → `setTasks` → o componente roda de novo

```
# servidor (EJS)
<div id="root"
  data-id-user="..."
  data-tasks="...">
# navegador (React)
initialTasks → useState
```

Fica para a próxima aula

useRef
valor que sobrevive às renderizações
como o estado, mas sem disparar
nova renderização

useEffect
código que roda *por causa* **de uma**
renderização
buscar dados, assinar eventos

Os dois são saídas controladas do ciclo estado → renderização → tela. Só fazem sentido com o estado sólido.

TF 08

React: Gerenciamento de Estados

- → Crie um **app React** (Vite + TSX): to-do de tasks que adiciona, marca, exclui e **filtra**
- → Só **estado, props e callbacks**: **sem** `useEffect` e **sem** `useRef`
- → Só `const` · booleano `isDone` · sem `else`

```
# branch = RA + -tf-08 (sem isso, o TF vale ZERO)
git switch -c 1234567-tf-08
# na raiz: diretório com o MESMO nome da branch
1234567-tf-08/
  tf.md ← nome, RA e GitHub
  app/ ← o projeto React
```

Entrega por Pull Request, título `TF-08 · RA · Nome`. Prazo: a definir · detalhes em TF08.md.